

ABRA — the plain-English deck

Version 5.205.0 · 2026-08-28 · Will Hooper

5.205.0 — WE PAUSED THE BIG SIMULATOR PUSH. HERE IS WHAT WE FIXED, AND — MORE IMPORTANTLY — WHAT WE STILL CANNOT TELL YOU.

For eighteen days we stopped updating these documents on purpose, so that fixing the simulator did not have to stop every hour to write about itself. Instead every fix wrote one line to a running log. That log had 274 lines in it when we stopped. The changelog has 233 releases from the same stretch, and there are 189 dated write-ups in the reports folder. The log is now deleted and the normal write-it-up-as-you-go rule is switched back on, which is the whole point of deleting it.

THE MOST USEFUL THING WE LEARNED IS NOT ANY ONE FIX. It is that the thing doing the measuring was wrong at least six times. Six separate occasions where we thought we had found a broken simulator and had actually found a broken ruler. One night we had thirty accusations against the engine; when we checked them properly, twenty-three were the test being written wrong, seven were the rule being written wrong, and exactly **one** was the game actually being simulated wrong. Another time, eighteen out-of-date certificates were being published as though they were a broken simulator. If you take one thing from this page: **before you believe the scoreboard, check the scoreboard.**

AND THE BIGGEST FIX MADE THE PROBLEM LOOK WORSE, WHICH IS EXACTLY WHAT IT SHOULD HAVE DONE. Deep inside the simulator is a dice roller. It is not real randomness — it is deliberately repeatable, so that we can play the same game twice and compare. The way it turned a situation into a number had a flaw: if two events differed only in "which hit of the move is this", it did not really roll again. It nudged the previous answer slightly. Two hits in a row landed in the same damage bucket **89.5%** of the time, when the honest number is about **6%**. The roller looked perfectly fair if you averaged it over a whole run, and was almost completely predictable one step at a time.

Because of that, our simulator and the real game had been agreeing partly by luck. They were only ever being compared across a thin slice of the things that can happen. We fixed the roller and the number of games where the two disagree went from **3 to 14**. Nothing got worse. We could finally see.

SO: EVERY NUMBER IN THIS DECK THAT WAS MEASURED BEFORE 27 AUGUST 2026 IS DEAD, NOT MERELY OLD. Not "a bit out of date" — genuinely not evidence of anything, because the comparison behind it was narrower than it claimed. We leave the old paragraphs in place, because this project does not quietly delete things it used to believe, but **none of them is a current result**. If you want to know where something stands today, run `node engine/status.js`. If that prints nothing for it, we do not know.

WHAT WE CAN TELL YOU TODAY

- **Every mechanic we have thought to test is tested, and they all fire.** 780 checks, 780 of them live, none missing. That is a laboratory result: we stage each thing deliberately, one at a time. It tells you *is this right*. It does not tell you *does this come up in real games* — that is a different scoreboard, and it is the one that is currently blank.
- **Damage is exact.** Six thousand comparisons against the real game, zero disagreements, and zero at each of the sixteen points of the damage range checked separately rather than lumped together. **But read the small print, because it is genuinely small:** that test covers damage only — no items, no abilities, no turn order, no switching — and it has **never once tested a move that hits more than once**. It skips all of those. Four multi-hit bugs were found and fixed this month and this test could not have seen any of them.

WHAT WE ARE REFUSING TO TELL YOU, AND WHY THAT IS THE HONEST ANSWER

Five of the eight boxes on our "is the simulator right yet" scorecard are **blank**. Not failing — blank. The measurements behind them were taken against a slightly different copy of the engine than the one now on disk, so those numbers describe a program that is not the one we are running. We could print them with a warning attached. We are not going to, because we have done that before and people quoted the number and skipped the warning — including us.

For the record, the difference is not a real change to the game. One data file got rewritten by a routine housekeeping step with different invisible end-of-line characters. Same content, different fingerprint. It still counts, and the only cure is to run the comparison again.

"THE BOARDS MATCH" MEANS 33 THINGS OUT OF 80. When we say our simulator's game state agrees with the real one, we are comparing 33 kinds of state. There are 80 kinds that a legal move, ability or item can create. Four more are openly declared as not compared. That leaves **43 that are in neither list** — not compared, and not admitted to be uncomparing — and 25 of those can be sitting on the board at the exact moment we take the comparison. A thing you forgot to compare looks precisely like a thing that agreed. This is the biggest caveat on the page.

AND THE FREEZE IS STILL ON. Everything that reads the simulator — how good our position-scorer is, the search results, the head-to-heads, the trained weights — remains withheld. The rule we hold ourselves to is: those numbers do not become true when the simulator becomes correct. They become **re-runnable**. Somebody has to run them again.

3.98.0 — QUICK GUARD IS SUPPOSED TO STOP FAST MOVES. OURS STOPPED NOTHING AT ALL. Some moves always go first — Fake Out, Bullet Punch, Aqua Jet. Quick Guard is the answer to them: it protects your whole side from anything that jumps the queue. We checked every way the game has of stopping a fast move, and four out of five worked. The fifth was Quick Guard, and it did not exist as far as our simulator was concerned — clicking it wasted your turn. The reason is small and embarrassing: Quick Guard and Wide Guard are described in our data with the *exact same four labels*, so the program was telling them apart by their names, which is the one thing we have a rule against. The information that separates them — one stops fast moves, the other stops moves that hit everybody — was already sitting in the data file and nothing was reading it. It reads it now. Quick Guard also correctly stops a status move that a Prankster ability has made fast, and Feint still punches through it, because Feint punches through every shield in the real game. Wide Guard was checked and is unchanged.

3.97.0 — FOUR MOVES DEALT THE WRONG DAMAGE BECAUSE WE ROLLED THEM ONCE AND MULTIPLIED. A move that hits three times should roll three times. Ours rolled once and multiplied the answer, which is fine when every hit is the same and wrong when it is not. Triple Axel gets stronger with each hit — we gave it the weakest hit three times, so it did **exactly half** its real damage. Dragon Darts fires one dart at each opponent — we fired both at one, so the partner took **nothing**. Beat Up is one punch per healthy team-mate, and we added their strengths together into a single punch. Fickle Beam has a 30% chance to go all out, and we gave it 30% of the boost every single time — a number the move never actually has. All four are fixed by giving each hit its own roll. Ordinary moves are untouched, and that was checked rather than assumed: every one of the 500 moves was played through a real turn before and after, and only these four came out different.

3.96.0 — THREE ITEMS DID NOTHING IN OUR SIMULATOR, AND NONE OF THEM WAS ACTUALLY MISSING. Iron Ball halves your Speed. Our engine handles that perfectly — for Choice Scarf. The rule that decides which items get the treatment was written as "the item called Choice Scarf" rather than "any item that does this", so Iron Ball was left out for 139 recorded uses while the working code sat there unused. Same story for Light Ball: the rule listed four items by name, and **all four are banned in this format** — so it could never do anything for anyone. And Oran Berry heals a flat 10 health, while we only knew

how to read heals written as a fraction, so we read "unknown" and did nothing — which is the right call, guessing would be worse. All three now work it out from the game's own rules instead of a list of names. **The lesson:** something can be built, correct, and working, and still be switched off for a particular Pokémon or item because a list somewhere never mentioned them.

3.95.0 — WE THOUGHT WE COULD KNOCK OUT A MIMIKYU. WE CANNOT. Mimikyu has an ability that soaks up the first hit it takes — the attack does nothing at all. Our simulator has two halves: one plays out the turn, the other just works out "how hard would this hit". The turn-playing half knew about it and has for a while. The damage half did not, so it kept saying Wood Hammer does 120 to a Mimikyu when the real answer is zero. Anything picking moves was reading the wrong half, so it believed it had a knockout that was not there. Both halves now ask the same single source. **This was a quarter of our "is the simulator correct yet" scorecard, and it was one row.** That section is now clean — 150 out of

1. **The tricky part was the fix, not the finding:** the turn-playing half gets its numbers from the damage half, so the moment we correctly said "zero", the turn half stopped noticing the ability had been used at all. Caught that before running it.

3.94.0 — A MOVE THAT HURTS YOU TO USE IT WAS FREE FOR US, ON TWO MOVES. Some attacks lower your own stats as the price of using them — Close Combat drops your defences, Draco Meteor drops your special attack. We handle all of those correctly, on more than twenty thousand recorded uses, which is precisely why nobody looked closer. It turns out the real game stores that price in **two different places**, and we only ever read one of them. Two moves keep it in the other place — Clanging Scales and Scale Shot — so for those two the drop simply never happened: the real game lowered the user's defence twice over two turns, ours left it untouched. Fixed by reading both places. **We also raised a much bigger alarm and then shot it down ourselves**, which is worth saying: it looked briefly like a label covering 22,000 uses had nothing reading it, including Close Combat. It does not — the engine gets there another way, and the six moves already testing correct are what proved it. A missing label and a missing mechanic are not the same thing.

3.93.0 — THE MOVES THAT SQUEEZE YOU FOR A FEW TURNS WERE ALL COUNTING A TURN SHORT. Bind, Fire Spin, Infestation, Sand Tomb, Snap Trap, Whirlpool and Wrap trap the target and chip its health for a few turns. All seven were off by exactly the same amount in exactly the same way, which is what told us it was one mistake and not seven. Someone had written down "4-5 turns" — the number a player feels — but the real game keeps a slightly different counter behind the scenes, starting one higher and ticking it down on the very turn the move lands. We now read that counter straight out of the real game instead of writing a number down. This is the third time this exact mix-up has bitten us, and the first two fixes missed this one because it is stored somewhere else in the code. **We proved it was broken before we proved it was fixed**, by running the old frozen copy of the engine and watching it get the wrong answer. Seven test rows went green and nothing else changed. Whole-game agreement did not move, and we are saying so: these moves are rare enough that they hardly ever decide where a game first goes wrong.

3.92.0 — WE WENT LOOKING FOR MORE OF THE SAME, AND TWO OF WHAT WE FOUND WERE REAL. Yesterday we caught a test using Tackle, a move this format does not have, so we checked every test for the same habit. Most were harmless. Two were not. One test needed three Pokémon to stand still and do nothing, and told them to use Splash — a move our simulator has never heard of. They stood still because the simulator could not find the move, not because the move does nothing. That keeps working right up until it does not. The other test only ran a check when the move "exists", and it turns out a banned move still counts as existing — so that check never once did its job, and the case ran on a move nobody in this format can use. Both now use moves that really are in the game, picked from the game's own list rather than typed by hand. No result changed, which is exactly what we expected.

3.91.0 — OUR TESTS COULD TEST THINGS THAT ARE BANNED, AND WOULD HAVE HAPPILY REPORTED THEY WORKED. When we test one small piece of the game, we build a fake Pokémon and hand it to the official simulator. It turns out the simulator never checks whether that Pokémon is legal. Hand it a banned item and it will play along, our engine will play along, the two will agree, and the test goes green — about something that can never happen in a real match. Will asked the obvious question: Showdown ships a team validator, why aren't we using it? Now we are. It is stricter than a ban list, because it also knows which moves each Pokémon can actually learn — it caught a test we wrote yesterday giving Meganium a Fire move it cannot learn. **We kept one deliberate exception, on purpose.** To compare several abilities fairly you put them all on the same Pokémon, even though that Pokémon could not really have most of them. That is the whole point of a controlled comparison, so it is still allowed — but it now has to be written down as a choice rather than happening by accident. Banned things are never allowed, no exceptions. **The first thing the new check caught was our own test file:** it had been padding empty slots with Tackle, and Tackle is not in this format.

3.90.0 — MOVES THAT HIT TWO TO FIVE TIMES ALWAYS HIT 3.1 TIMES, WHICH IS NOT A THING THAT CAN HAPPEN. Rock Blast, Bullet Seed, Icicle Spear and the rest roll for how many times they hit. Our simulator never rolled: it used the average, 3.1, every single time. So half these moves came out too strong and half too weak, and it looked like eleven separate small bugs instead of one. It was one. The simulator now rolls the count the same way the real game does, and the moves that always hit exactly twice — Double Hit, Dual Wingbeat, Twin Beam — were checked to make sure they did not move, because a "fix" that breaks the simple case has broken something else. Three moves in the group are still wrong and each is a different problem: Triple Axel hits harder with each hit, Scale Shot changes your own stats afterwards, and Dragon Darts spreads its two hits across two opponents. Those are named rather than lumped in.

3.89.0 — SOME POKÉMON GET STRONGER WHEN YOU HIT THEM, BUT ONLY IF YOU HIT THEM THE RIGHT WAY. Anger Point maxes out its Attack when it takes a *critical* hit. Justified only reacts to *dark* moves. Weak Armor only to *physical* ones. Our simulator gave all of them their bonus on every single hit, so it thought hitting these Pokémon was far more dangerous than it is. The one member of the group that genuinely has no condition — Stamina, which is also the only one people actually run — was right the whole time, and that is exactly why nobody spotted the other eleven. Separately, four healing moves (Synthesis, Moonlight, Morning Sun and Strength Sap) did **nothing at all:** the Pokémon spent its turn and recovered zero health. In a sandstorm that was worse than doing nothing, because the sand still hurt it. All fixed, with tests that fail if either comes back.

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — ONE POKEMON CARRIES ITS OWN SUNSHINE, AND HALF THE ENGINE COULD NOT SEE IT. Meganium's mega form fights as if the sun were out, even when the sky is clear. Weather Ball is a move that changes type with the weather: Normal in clear skies, Fire in sun. Our damage maths knew about the private sunshine and called the move Fire. The part of the engine that decides whether a move is allowed to connect did not, and called it Normal — and a Normal move does nothing at all to a Ghost. So the mega's

signature attack dealt zero. Both halves now ask the same question of the same function. We checked it against the real game first: 0 damage without the mega, 97 with it, and exactly the same 97 under a real sun. Live mechanics went from 325 to 326, and nothing else moved.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — FOUR OF THE COMMONEST ABILITIES IN THE GAME HAD NEVER ONCE WORKED, AND THE SIMULATOR WAS RIGHT TO IGNORE THEM. Blaze, Torrent, Overgrow and Swarm all do the same thing: when the Pokémon drops below a third of its health, its attacks of one type hit 50% harder. Our simulator knew the 50%. It did not know "below a third" — that part was written down as an English sentence, and code cannot read a sentence. Faced with a rule it could not check, the simulator did the safe thing and did nothing, which is right: guessing the cut-off would have handed every Charizard a permanent 50% bonus. So the fix was to write the rule down as a number the code can check, taken straight from the official game's own source. The catch was the exact line. "A third of 150" is 50, and if you compute it the sloppy way a computer gets 49.999999999999993 — so a Pokémon on exactly 50 would miss out on a boost it had earned. That single point of health is the one that decides games, so the test now checks precisely there, in both directions, against the official engine.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all` -stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at `docs/` and `web/` but never at `app/`, which is the copy a visitor actually loads. Five withheld verdicts were being published from `app/` the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one

needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: PRE-CHANGE had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body Serious while the stored sheet beside it said Modest, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them ("evs": null on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in clearVolatile, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the

thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS

EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT.

Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT.

Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE TEST ITSELF COULD SEND THE TWO COPIES OF THE GAME DIFFERENT

POKEMON. When the test tells both engines to switch, it names the Pokemon it wants. One copy looked that name up one way and the other looked it up another way, and the two only agreed while nothing had been renamed — which stopped being true the day we taught the engine that Mimikyu becomes Mimikyu-Busted and Palafin becomes Palafin-Hero. Worse, when either copy could not find the Pokemon it simply did nothing and said nothing, so one side could switch while the other stood still and the difference looked like an engine bug. Both now ask the same question, and any failure to find it is counted out loud.

THE ENGINE FOLLOWED THE POKEMON, AND THE GAME FOLLOWS THE SPOT IT WAS

STANDING IN (3.77.0). If you aim at the left-hand Pokemon and it switches out before your move goes off, your move hits whatever walked in. Our simulator kept aiming at the Pokemon — sometimes at one that was already on the bench — so stat drops from Charm and Parting Shot went nowhere. Ally Switch, which swaps

your two Pokemon between spots, did not exist at all: the simulator treated it as a wasted turn. And Speed Boost was handing out its Speed one turn too early to anything that had just come in. All three are fixed, each proved by playing the same turn in both simulators and comparing every field of the board.

A COIN FLIP WE HAD BEEN CALLING THE SAME WAY EVERY TIME (3.74.0). When two Pokemon are exactly as fast as each other, the real game flips a coin. Our simulator did not — it always let the same one go first, and it had done that since the day it was written. That matters more than it sounds: nine out of ten Pokemon in this format share their Speed with some other Pokemon, so this was not a corner case, and it was not just a test problem — it is the same code that decides the order in every game the bot plays. Fixing it was not a matter of flipping the answer round. The official engine sorts in a way that shuffles ties, and the tempting shortcut — "always let the second one go" — would have made our test go green while the bot stayed wrong in real games. We copied the real rule instead, so both engines land on the same Pokemon for the same reason. Alongside it: Palafin was transforming at the wrong moment, Mimikyu's broken disguise never changed its name on the board, and a Pokemon that switched out mid-attack was paying its own damage costs after it had already left. Twelve more abilities and moves that nobody had ever tested now have tests. Two of them turned out to have been working all along.

WE WERE ONLY EVER TESTING ONE CORNER OF THE GAME (3.73.0). To compare two engines fairly you freeze the luck, so both get identical dice and any difference has to be a real bug. We froze it to a single setting — and that setting meant the speed tie always went the same way, every move under 100% accuracy always MISSED on both sides, and damage was always the maximum roll. Rock Slide had never once connected in the entire history of this test. It does now. There are four frozen settings instead of one, so we look at four corners rather than the same corner four times, and a comparison between runs with different settings is refused rather than quietly reported. We also stopped counting a move as tested just because the bot clicked it: clicking Haze when there are no stat boosts on the board does nothing at all, and the old rule marked Haze covered and moved on. Five mechanics turned out to have been "covered" that way while doing nothing. **Every score published before this is measured on a different set of games and cannot be compared with what comes next.** And one real engine bug came out of it: when two Pokemon have exactly the same Speed, the real game and our copy have been picking different ones to move first — every time, for as long as this test has existed.

ROADMAP #92 — THE DAMAGE-STAGE CLASS. FOURTEEN MULTIPLIERS WERE APPLIED AT THE WRONG STAGE AND FIVE WERE ABSENT (3.73.0). Showdown applies each multiplier at a STAGE — a base power, a stat, or the final damage — folds every handler at that stage into ONE modifier, and spends it ONCE. This engine applied about a third of them a stage late, and separately: Black Glasses on the final damage where the authority puts it on base power reads 109 against 108. That one-point shape is why it survived every existing check — both engines "apply Black Glasses", so the census saw it LIVE, the interaction matrix compares a ratio between arms, and the damage differential allows a 12% midpoint band by design. LANDED: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force, Rising Voltage, Dry Skin and the -ate x1.2 into ONE base-power relay spent once; Thick Fat (73% wrong), Heatproof, Purifying Salt and Water Bubble (77%) into the STAT relay, because they modify a stat and not the damage; Helping Hand (wrong on 5 of 5 audited rows) and Friend Guard (21.4%) off the hit site and into the chains they belong in; Sniper out of the crit's plain multiply and into the final chain; and the rolled crit's POSITION into the damage range before the randomizer, where it was 46.5% wrong at the bottom roll and invisible at the top one every check pins. The four FIELD terrains were absent entirely — a Grassy-Terrain Earthquake was priced at DOUBLE the real number. New gate `tests/test-damage-stages.js` is **1,728 of 1,728 exact** against the authority across all sixteen damage rolls and both crit states, and was shown RED on two deliberate

reversions before being trusted. `damageBoost` is still NOT wired as a class and the reason is a property of the tag: it carries neither the stage nor the condition, so wiring it would hand Blaze a permanent x1.5 on 5,808 sheets. Census 293/294 → 298/299 live.

ROADMAP #81 WIRE 12 — FIVE ENGINE DEFECTS OFF THE TURN-1 BOARD, TWO OF THEM MIS-DIAGNOSED BEFORE THEY WERE FIXED (3.71.0). The auras (Fairy, Dark, Aura Break) are wired FIELD-WIDE at the base-power stage — they multiply one type for every body on the field, the foe's moves included, and Aura Break INVERTS to x0.75 rather than cancelling; exact against the official engine on 12 of 12 staged arms. Baton Pass and Shed Tail switch for the first time (`passesState` had been derived and never consumed, so Baton Pass was a no-op turn and Shed Tail paid half its user's HP to stand still). Curse is two moves and the engine had neither. Perish Song counted from 3 instead of 4 and therefore fainted every affected body on both sides a full turn early, on 1,141 corpus uses — the KO itself had always fired. And ROADMAP #81 WIRE 10's measured board regression is one line: the Life Orb toll was being paid by a move that MISSED. **Two of the five briefed diagnoses were wrong** — the tagger was not testing `selfSwitch === true`, and the substitute doll was not confounded, it was a regression this project introduced at WIRE 7 on a misquoted source line. Census 281/282 → 293/294 live.

THE INSTRUMENT WAS MEASURING ANNOUNCEMENTS, AND THE HEADLINE IS NOW THE BOARD AT THE END OF TURN 1 (3.70.0). `engine/board_state.js` reads HP, status with its counters, items, all seven stat stages, aliveness, every field condition WITH ITS CLOCK and the persistent volatiles out of BOTH engines' live bodies at every turn boundary, after the whole residual phase. Read every figure from `data/state-ladder.json`. **The board at the end of turn 1 is identical in 56.0% of games at the pre-WIRE-1 baseline (1119/1998) and 66.9% at the top rung (1337/1998)**, peaking at 69.3% at WIRE 9; whole-game board agreement went 6.4% → 15.6% against a protocol number that read 1.8% → 10.3%, so the wires were real and the protocol number overstated them. **WIRE 10 is a regression the protocol instrument scored as an improvement** — 47 fewer clean turn-1 boards, and diffed per field it is one field, end-of-turn-1 HP wrong in 427 → 473 games. **41.0% of games whose narration parted inside turn 1 reached an identical board anyway.** The comparator proves itself first: 7 representation mappings red-demonstrated in both directions and 25 planted state divergences, each of which must be caught at the planted boundary and localised to the planted field — 25/25 on all fourteen arms.

What changed (3.69.0): we asked whether making the simulator more correct makes the bot predict better, and the answer is no.

A night of engine fixes made our simulator agree with the official one for much longer before the two part company. The obvious next question is whether that helps — and nobody had checked. So we took the one number the bot actually uses to choose a move (its guess at "how likely am I to win from here"), and we ran it over **8,883 real ladder games twice**: once through the old engine, once through the fixed one, on the same games with the same dice. The two engines differ in exactly one file, so nothing else can explain a difference.

They scored the same. Not "close" — the same, to four decimal places, with an error bar tighter than the smallest difference the test could have spotted. On the games where both engines committed to a winner, the fixed engine got 37 right that the old one missed, and the old one got 36 right that the fixed one missed. That is a coin flip.

We also checked it position by position: are the games where our simulator is most faithful the games where the bot predicts best? **No relationship at all** — and we proved that is a real finding rather than a blunt instrument, by re-measuring the faithfulness a second way and confirming it lines up with itself.

So what IS wrong with the bot's guess? It is wildly overconfident. It uses the full range from 6% to 94%, but reality only moves between 44% and 59%. When it says "94% sure", it wins 59% of the time. That is the thing to fix, and no amount of further engine work touches it.

The honest summary: the engine fixes were real and worth having, and they bought nothing for the decision the bot makes. We are saying so rather than quietly moving on.

What changed (3.68.0): we went back and checked what a night of engine fixes was actually worth, and the honest answer is less than it looked.

One night we found and fixed six things wrong with our engine, and each fix reported a before-and-after number. Then we discovered the before and the after had not been measured on the same set of games, so none of those numbers meant anything. We had kept a frozen copy of the engine at every step, so instead of guessing we replayed all nine of them on one fixed set of 1,995 games.

The headline is a negative one and it is the important one. **The typical game still falls apart after a single turn, exactly as it did before any of the six fixes.** Games that match the official engine all the way through went from 2 to 22 out of 1,995. What did improve is how far into a game we get before the first disagreement — the average roughly doubled — but "further before it breaks" is not the same as "it works", and we should not let the second number stand in for the first.

Two things the replay found that the original before-and-afters had got wrong. **A change nobody wrote up as a fix at all turned out to be worth more than one of the numbered fixes** — it happened to sit just before it, and the pairwise comparison had quietly credited it to its neighbour, which then looked twice as good as it was. And one fix that is definitely correct — an arithmetic rounding error, real and worth fixing — changed **none** of the 1,995 games. Being right and being measurable here are different things, and it is worth saying so out loud rather than rounding it up.

We ran the starting point twice, first and last, with eight replays in between. It gave identical answers, so the differences above are the engine and not the measurement wobbling.

It plays the same real game twice — once in our engine, once in Pokémon Showdown's own — and stops at the first place they disagree. On the first proper run, **159 of 160 games disagreed somewhere**, and most of them within a single turn. That sounds alarming and it is exactly what we wanted: for the first time we can see the gap instead of guessing at it, and every disagreement is a specific, fixable thing rather than a worry.

Two honest caveats we keep attached to that number. It has not yet tested anything past the first turn. And it tested **no mega evolutions at all**, which is about a quarter of what this format actually plays — that is the next job, not a footnote.

The first thing it caught was a claim we had made ourselves a few hours earlier, and it proved us wrong. That is the whole point of building it.

A slide-by-slide, jargon-light tour. The white paper (linked on the last slide) has the math and sources.

Slide 1 — ABRA

A family of small AI models for competitive Pokémon (Champions VGC). Built from thousands of real ladder games. Honest about what it can and can't know.

Slide 2 — The big idea

You **can't reliably predict who wins** a match from the two team sheets — in this format it's close to a coin flip, and even a strong player-rating model barely beats a coin. So ABRA stops trying to call the winner and instead **helps you make better decisions**: what to bring, and what to do turn by turn.

This is the same move sports analytics made with "expected goals" — you can't predict the final score, but you can measure the value of each shot. ABRA is that idea, pointed at Pokémon.

Slide 3 — How it's built

One rule: **keep every real game, and analyse on top of it**. ABRA saves every public ladder replay into a growing library and builds its models on that library — so it gets smarter as more games come in, and never has to re-download anything. It runs on a normal laptop; no special hardware.

Slide 4 — The town of models

Each model is a "house" you can visit on the site:

- **MEDICHAM** — the damage engine. Matches the community standard, but disagrees with the game's OFFICIAL engine by a wide margin, so it is being replaced by that engine and kept only as a lookup.
 - **MEW** — plays the official engine against itself, so we are not limited to the games people happen to upload.
 - **GURU** — reads the metagame: which team styles beat which, from real results, with error bars.
 - **XATU** — reads the opponent: the likely item, ability, and moves behind each Pokémon.
 - **PORY** — mid-battle win chance. **Retracted as a finding:** it works out to counting how many Pokémon each side has left and how healthy they are, and it does not beat exactly that baseline.
 - **The scoring bot** — the part that finally *looks at the board*.
 - **SLOWKING** — the strategist: there's no single best team, so it plays a smart mix.
 - **CHOMP** — the team picker: which four to bring, which two to lead.
 - **ALAKAZAM** — the coach (in progress): the best move to make, right now.
-

Slide 5 — The win: the practice opponent can finally see

Everything ABRA claims about "this build beats that build" comes from playing the game out against a practice opponent. That opponent used to pick its move purely by **how popular the move is** — it had no idea what was standing in front of it. So it fired attacks at Pokémon that were flat immune, it used moves that couldn't possibly work, and when there were two enemies to aim at, it picked one by flipping a coin.

We fixed it by **watching what people actually do**. We took about 48,000 real decisions from games where both players' full teams were public — so we could see not just what someone clicked, but everything they *could* have clicked — and learned how much a real player's choice depends on what's in front of them. Nobody wrote the rules; they were measured.

The result, checked on games the learning never saw:

- it now hits the enemy's weakness **half again as often** as before,
- it wastes **a third fewer** moves on things that simply cannot work,
- and it almost never fires at something immune any more.

It is still short of a human on all three, and one thing it still does badly is decide **when to switch** — that's the next job. But the practice opponent is no longer playing blind, which means the numbers it produces are worth more than they were.

PORY also beats a coin flip mid-game, using how many Pokémon each side has left and their health, and it's *calibrated* — when it says 70% it's really about 70%. It's live on the site as a per-turn "you're at X%". **Read the retraction on slide 4 with this, and not after it:** `data/pory-eval.json` ends its own verdict with the sentence "*It does beat a coin ... and neither of those is evidence of a learned value function.*" Beating a coin is not the bar; PORY ties a model that only counts Pokémon and health, which is the same two things this

paragraph credits it with using. The calibration is real and the *learning* is what was withdrawn. (Added 2026-08-22: this slide previously stated the win and the calibration with the retraction two slides away, which reads as a result.)

Slide 6 — The honest parts (this is a feature)

Two things we tested and reported straight, even though they're negatives:

- **Picking the team doesn't beat a coin.** We tested whether the team-picker's choices track who wins — they don't, more than chance. So we don't oversell it. The damage math is still exact and useful.
- **The "rock-paper-scissors" metagame is a hint, not a fact.** The data suggests Trick Room beats Hyper Offense beats Sand beats Trick Room — but each of those rests on only ~13–18 games, so the error bars are wide. We label it suggestive, and it'll firm up as more games arrive.

Added 3.32.0 — three more, and the first one is the worst thing we've found.

- **We thought Rage Powder was deleting attacks. It wasn't — and catching that is the story.** The test we wrote to check it aimed a *Dragon*-type attack at a Pokémon that is immune to Dragon. The redirection worked perfectly and moved the attack onto something that takes zero damage, so the test saw zero either way and we believed the wrong thing for several hours. Re-run against a Pokémon that can actually be hurt, the untouched code works fine. Nothing was broken and nothing we had measured was invalidated. **A test can be wrong in exactly the shape of the bug it is looking for** — that is now the ninth time here, and the first time it fooled us.
- **A real one next door, though.** Lightning Rod and Storm Drain — abilities that pull attacks in — genuinely weren't working, 1,901 uses. Fixed.
- **The number we quote for "how often we disagree with the real game" wasn't repeatable.** It picked its test matchups at random and never wrote down which ones. Two runs on identical code gave 6 and then 3. It now uses a fixed list. (*Corrected 2026-08-22: this used to end "and the honest figure is 4 disagreements in 400". That test has been rebuilt twice since — it is far larger, and it now checks the whole spread of a damage roll rather than only its two ends — so the old score is not comparable and is not restated. The current one is printed by `node engine/status.js`.*)
- **The part of the bot that judges "am I winning?" — the figures are WITHHELD as of 2026-08-22.** This bullet used to give the reliability curve in plain words: what the bot wins when it says it is almost certain, and what it wins when it says it is almost lost. Those come from a measurement made by playing games in our own simulator, and that simulator is currently known to be wrong in ways we are still fixing — so every number that came out of it is set aside until it is re-run, rather than printed with a warning label next to it. Two things in this bullet do *not* depend on the numbers and still stand: the earlier reading rested on far too few games to support any claim either way, and the version being measured was not the one the bot actually uses.

Added 3.33.0 — we went looking for the same mistake everywhere else, and found it twice more.

The 3.32.0 note above says a result we'd published as a pass turned out to be unreproducible. Here's *why*, because the reason is more useful than the result: the file we kept recorded the answers and not the settings. It's like keeping a stopwatch time and not writing down which race it was. Two runs with very different settings produced files that were identical, and we could not tell which one we had.

So we checked the other three tests in the same series.

- **"The new search picks a different move much of the time" is a headline with its control missing.** A search that's just guessing disagrees with *everything* — including with itself. So the test

measures that too: run the same search twice with different dice and see how often it contradicts itself. That number is the yardstick, and the test printed it to the screen and never saved it. On an early version the yardstick was *higher* than the headline. Believing and having measured are different things, and it's the second one we publish. *(The headline percentage is withheld as of 2026-08-22 — it came out of the simulator that is currently being fixed. The point of the bullet is the missing yardstick, which does not need it.)*

- **"A single look-ahead costs N milliseconds" was timing something the bot doesn't do.** The measurement quietly used the settings the code falls back to when you don't say, and the bot uses different ones — a randomised playout at triple the length. Not wrong arithmetic; the wrong thing measured. *(The timing itself is withheld as of 2026-08-22, same reason as the two bullets above.)*

Every one of these tests now writes a small companion file recording exactly what it did: how many rollouts, what randomness, what horizon, which version of every file it read, and whether anything was uncommitted at the time. Older results get one reconstructed from the commit that carried them, labelled as a guess rather than a record.

Added 3.34.0 — the clearest example yet of how this goes wrong.

We built a page showing every model, published it, and asked for a review. It was completely broken — one stray quotation mark meant none of it could run, so it showed a title and a blank screen.

Two automated checks looked at that page and both gave it full marks. One counted twelve model cards and found twelve. The other checked that every number on it cited a source, and scored it 100%. Neither of them ever *ran* the page — they read it the way you'd read a document. So a page that could not work passed every test we had.

A person opening the link found it in about two seconds.

The same day, a second version of the same problem: two new pages were written, tested, saved and published to the code repository — and never copied to the folder the live website actually serves. The check meant to catch that compared one specific file, so a *missing* file was invisible to it. Again: found by opening the URL.

Both checks now exist in a form that would have caught them, and both were written the same way — by asking what the check could not see, rather than what it did see.

Saying what we *can't* prove, as plainly as what we can, is the whole point.

We also caught ourselves twice this week, which matters more than any single fix. A result we'd published as a pass — "playing the position out beats just counting what's left" — turned out to be unreproducible from anything we'd kept, and on the evidence that survives it's a tie, not a win. And a claim written in our own notes at 3am about an outside dataset was wrong within four hours; it's corrected in place with the reason, not quietly deleted.

Slide 7 — What's next

The one lesson that changed the plan. We spent a day teaching the bot more about Pokémon — four separate things it didn't know before. All four made no difference at all. Then we changed *what it was trying to do*, twice, and both times it got a lot better.

Teaching it more facts had stopped helping. Changing its goal was what worked.

The reason is simple. The bot was trained to **copy what humans click**. That is a different goal from **winning**. It even shows up in the numbers: the model rates "use a spread move that hits both of them and doesn't hurt my own partner" as a *bad* idea — not because it is bad, but because humans rarely click it. A bot

told to avoid its own best plays will avoid them.

- **Teach the two Pokémon to act as a team.** We already built this once. It plays worse, and now we know why: it was taught to *resemble* people rather than to *win*. Retraining it on the right goal is the single most promising thing on the list.
- **Fix a small unfairness in the training data.** When a Pokémon is locked into one move, we were still showing the bot all four as if it had a choice. It affects about 1 in 15 items.
- **Find out how readable it is.** We have no answer to this. An old figure said a rival bot built to beat ours won 63% of the time; that number is **withdrawn** — it was measured on a third of the features the bot now uses, on a dirty set of games, before most of the engine was fixed. The re-run was ruined when the thing being tested got rebuilt halfway through. So the honest position is that the question is open, not that the answer is bad. A bot can get better on average while staying just as easy to read; those are different questions and only one has been checked.
- **Then, looking ahead.** Right now the bot only thinks about *this* turn. Looking one turn further is a real project, but it's now measured rather than guessed: there are only **72** sensible combinations to consider each turn, not the trillions people assume.
- **ALAKAZAM** — the in-battle coach. A light version runs in your browser; a version that could beat top humans needs a rented cloud computer and months of the AI playing itself.

Something we got wrong, kept here on purpose. For two days nobody updated these documents while the code kept changing. The next person to read them — which was us — then described several models completely incorrectly, including calling one "not built yet" when it had been built, tested, and measured. Documents that lag the work don't just go quiet; they actively mislead.

Slide 8 — You cannot measure something while you are changing it

Four teams work on ABRA at once. That is deliberate — it is why the work was split four ways.

One night all four were running. Their files were kept separate so nobody could overwrite anybody. A 7,100-game experiment — "can anything beat our bot?" — was destroyed anyway.

The reason is simple once you see it. Halfway through the experiment, another team **improved the bot being tested**. So the first half of the experiment played against the old bot and the second half played against the new one. The final number described neither.

Nothing broke. No error appeared. The result looked completely normal, and the automatic checker approved it — because that checker was asking "*is this file newer than the thing it came from?*" and the answer was yes. A file can be newer than something it never actually read.

The fix is not to make the teams take turns. That would throw away the whole reason for having four of them.

Instead, an experiment now works from a **photograph of the bot** rather than the bot itself. Before measuring, we freeze a copy of everything that could change the answer, and the experiment reads the frozen copy. Other teams can rewrite the real thing all night; the experiment never notices.

The honest consequence: **we currently have no answer to "how beatable is our bot?"** The old answer was measured on a version of the bot that no longer exists, and the new attempt is void. Saying so is better than quoting a number we cannot stand behind — that number had been called the most important one in the project.

One thing did survive, and it is genuinely useful: when the bot plays a mirror match against itself it wins 49.7% of the time, which is a coin flip. That confirms our testing setup does not quietly favour one side — a worry an earlier, smaller sample had raised.

Slide 9 — Practising in different conditions from the match

We found something bigger than the bug we went looking for.

In Champions, players often reveal their whole team before the battle. Our bot **uses** that information when it plays. But when we *taught* the bot, we were throwing half of it away — the bot learned without knowing the opponent's abilities or moves, then plays with both.

It affects **half of every decision** it was trained on, and virtually every game. It is like training a driver in fog and then handing them the keys on a clear day: not obviously worse, but they never learned to use what they can now see.

We have not fixed it yet, and that is deliberate. Fixing it means retraining everything, and there is a real question first: sometimes opponents *decline* to reveal their team. A bot trained to depend on information that is sometimes missing can fail badly when it disappears. We have made that mistake before, in exactly this shape, and it is worth one deliberate decision rather than a quick fix.

Update (version 3.40.0). The decision was made: the bot plays with the full team sheet, always, and the rarer no-sheet games are set aside for now. The first half of the retraining is done — the move-picking layer now learns from the same information it plays with, and we *counted* that the information actually arrived during training (99.7% of decisions) instead of assuming it. The second half (the layer that picks pairs of actions) is queued next. No claim yet that the retrained bot is better — that comparison is set up against a frozen copy of the old one and runs next.

Also in this pass: the simulator learned eight more rule families and now covers 181 of the 186 mechanics we probe for, with the remaining 5 named and explained; and we compared our two separate "rulebook" files fact by fact — they disagreed exactly twice, and one of those (how often Iron Head flinches in this format) was a real bug the bot had been playing with. It reads the right number now.

Update (version 3.41.0, same night). The second half of the retraining is done too — the layer that picks pairs of actions now learns from the full team sheet, and we measured what all that sheet information is actually worth by replaying 45,000 real decisions with and without it. Honest answer: the bot judges positions measurably better with the sheet, but we cannot yet show it clicking different moves often enough to prove a win-rate gain — the effect is smaller than our measurement noise. We say it exactly that way rather than rounding up. Meanwhile the simulator's coverage grew again (202 of 205 probed mechanics, 3 remaining, each with a written reason), the mirror-match test hit 100% agreement with the official engine, and two real bugs — one in how Intimidate interacts with certain abilities, one where Sheer Force was hurting its own holder — were found and fixed with proof.

Update (version 3.42.0) — we were teaching the bot moves nobody ever picked.

Will said it first: *"i def dont like just tossing turns because they got outplayed with a move like Encore or Follow Me, these are the basis of VGC."* He was right, and it turned out to be worse than tossing them.

When Encore hits you, the game **takes the controls away** and makes you repeat your last move. The replay just shows you using that move. Because it is a move you legitimately own, every check we had said "looks like a normal choice" and the model dutifully learned it as one. The same thing happened whenever Roar or Whirlwind blew a Pokémon in — the replay writes that arrival exactly like a voluntary switch, so we were teaching the bot to make switches the player never made. **1,336 of 241,927 recorded actions across**

8,942 games were things no human chose. That is not a small rounding error being fed to the model; it is a small amount of outright fiction, and it was concentrated on the turns where the opponent had just outplayed the player. Those are gone now, and counted, so we can see the number.

The other half — Follow Me and Rage Powder — is subtler. The replay only records where an attack *landed*, so when a Pokémon soaks the hit we could not tell which target the player was aiming at. We used to write down "they aimed at the soaker", confidently, which was usually false. Now the model is told the honest thing: it was one of these two, we do not know which, learn accordingly. There is a standard statistical method for exactly this, and we tested it first on data where we had hidden the right answer ourselves — it recovers 97% of the damage when the problem is severe.

And here is the part we are not going to round up. Removing the fiction worked: on those turns the model now puts measurably less confidence in the move nobody picked. The Follow Me fix bought **nothing we can measure** — and our own test had predicted that before we ran it, because those turns are only about 1.3% of the data and the ambiguity is only ever between two targets. Overall accuracy did not move at all. We are doing it anyway, because a wrong label is a wrong label and the model has no way to tell us which of its mistakes came from one.

And then the simulator changed underneath both of those numbers, so we checked them again (3.47.0). A measurement is only worth the version of the code it was taken on. Two of these results had been computed with a simulator that changed the same day, which normally means the results have to be thrown away and re-earned. Before doing that we asked the cheaper question: did the change actually alter any of the 58 things the model looks at? We ran every one of them through the old simulator and the new one, on 1.75 million real decisions, and got byte-for-byte the same answers — so the change was real in the game engine and invisible to the model. We re-ran both measurements anyway, on a corpus that had grown by nearly three hundred games, and every conclusion above came back the same.

One more thing we found and did **not** fix: when a Pokémon is Taunted or Encored its menu shrinks, sometimes to a single legal option, and we were still scoring the player as though they had picked that move out of nine. That is about 1.6% of actions. It is a real problem, it is a different problem, and fixing it means retraining everything again — so it is written down with a number attached rather than quietly bundled in here.

Slide 9b — The rule was there. It was just pointed at everybody. (3.44.0)

Psychic Terrain is a floor that stops fast moves — Fake Out, Sucker Punch, Extreme Speed, the moves that go first. Our simulator knew that. What it did not know is that **the floor only protects Pokémon standing on it**. A Talonflame is in the air. A Pokémon with Levitate is in the air. Fake Out hits them straight through the terrain, and we were blocking it.

Fake Out is the single most-clicked move in this format — nearly thirteen thousand uses in our corpus. So this is not a corner case; it is a rule the practice opponent got wrong in a large slice of the games it plays.

Two things about it are worth more than the fix.

The first: the question "*is this Pokémon on the ground?*" was already answered in three separate places in the same file, by hand, and **the three answers did not agree with each other**. One of them was even keeping a counter of how often it knew it was giving the wrong answer. That counter had been running since the previous release, next to a note saying the information needed to fix it was not available — and the information had arrived a release earlier. There is now one function, and the four places that need the answer all ask it.

The second: our own test suite could not have caught this. The existing check for Psychic Terrain aims the blocked move at a **Garchomp**, which is standing on the ground, so it passes whether the rule is right or wrong. Every instrument we own asks *"does this mechanic happen?"*. None of them asked *"does it happen only where it should?"* — and that is the fourth bug of exactly this shape in two days. The new check has five arms, and one of them exists purely to catch a mistake we nearly made: Orthworm's Earth Eater makes it immune to Ground attacks, which looks in our data exactly like Levitate, and Orthworm is very much on the floor. We asked the official game engine and it told us so.

Slide 9b — We said we had 97 bugs. We had none of them.

A tool of ours changes a fact the simulator is supposed to read, then watches whether the simulator's behaviour moves. If nothing moves, the simulator was not really reading that fact. It flagged **97** of these, and they were passed on as 97 bugs.

Will asked what the top two actually were. **Both were fine.** Life Orb's damage boost is read from the data; only its recoil is written against the item's name, which is untidy but not wrong today. Light Screen's number is ignored **deliberately** — the data file carries the singles-format value, and this is a doubles engine where the real number is different. Using the data file's number would have made every Light Screen look a third better than it is.

The tool could see that nothing moved. It could not see **why**. It now answers that too, by reading the simulator's own source code rather than anybody's comments about it, and sorts every finding into four boxes: nothing implements this at all (the real bugs), the engine substitutes its own value on purpose, the engine works off the name instead of the data, and the test simply never reached that branch. **None of the 97 is in the first box.** The count we now track is that first box only — 163 items, none of them from the 97 — because a number that includes false alarms is a number people stop reading.

The rule was checked against three cases we had already worked out by hand, and it refuses to publish anything if it cannot reproduce all three.

Slide 9c — Everything that changes accuracy was switched off, and one move was charged for and never delivered (3.56.0)

Some moves miss. Some things make you miss less: Coil sharpens your aim, a Wide Lens is an item that does the same. Some things make you miss more: Sand Veil hides a Pokémon in a sandstorm. And No Guard means nobody ever misses at all. All four are in the game we are modelling, and together they show up about five thousand times in our replay store.

We measured each of them the boring way — play it out with the thing, play it out without the thing, compare. **All four returned exactly the same number both ways.** Not close, not within noise: identical, to the digit. Something that changes nothing at all is not a subtle bug. Every one of them was simply not connected.

Three separate reasons, which is the part worth noticing. A translation table turned the words "accuracy" and "evasion" into nothing, so a move that was supposed to raise three of your stats quietly raised two. Items and abilities were never consulted for accuracy anywhere. And the function that decided whether you hit **was not given the attacker or the target** — it was handed a move name and asked to answer a question that depends on who is holding what. It could not have got that right even in principle.

In the same pass: **Substitute took a quarter of your health and gave you nothing.** The doll was paid for and never built. That is 1,976 clicks in the store of a move that was strictly worse than doing nothing — the simulator would tell you the cost and then hand you an empty box.

The tempting fix for Substitute was the bigger bug, and we measured it before writing it. Some moves go *through* a Substitute — sound-based ones do, famously. It would have been natural to write the rule as "sound gets through". We checked: the three moves that most often need to get through in this format are Encore, Taunt and Disable, together about seven thousand uses, and **not one of them is a sound move**. A tidy rule would have walled all three.

Coverage is now 231 of 232 probed mechanics. The one that remains has a written reason, as do five more rules we know are absent — the largest of those needs information the damage function is never handed, so wiring it is a design decision rather than a fix, and it is written down as one instead of being quietly patched.

Slide 9d — The simulator could not say what it did, only where it ended up (3.58.0)

The problem. We check our simulator against the official one by comparing *results* — how much damage, what the board looked like after a turn. When they disagree, that tells you *that* something is wrong and almost nothing about *what*.

What the official simulator has that we did not. A running commentary. Every decision it makes it writes down, labelled: *Incineroar used Fake Out. Garchomp's Attack fell. The Focus Sash broke*. It is a transcript with the reason attached to every line.

What changed. Our simulator writes the same transcript now, in the same format, off by default so it costs nothing when nobody asks. The list of things it can write down was **read out of the official simulator's source code**, not typed from memory — and it checks itself two ways: it fails if we claim to write something the real one never writes, and it fails if the real one writes something we neither write nor have an explanation for. Ninety-one kinds of event exist; we write thirty-six, and the other fifty-eight each have a sentence saying why not.

What it caught on the first night. Two things, and both are about our own measuring tools rather than about the game:

- Our damage test only ever compares the *lowest* and *highest* possible rolls. It never looked at the fourteen in between — and in between, the two simulators do not agree about how likely each one is. We had been reporting 149 out of 150 correct on a comparison that could not see the middle.
- Our simulator does things in a different *order* inside a single attack than the real one does. The board ends up identical, which is exactly why the old test said everything was fine.

Neither is fixed yet, on purpose. Changing how a damage roll is picked would move every measurement this project has ever recorded. The point of tonight was to build the instrument that can see it.

Slide 9e — Somebody built a bot that beats a pro. Any strong human learns to beat it in five games. (3.62.2)

The result. A university team (Angliss, Cui, Hu, Rahman and Stone, published at AAMAS 2026) built a Pokémon doubles bot the proper modern way: show it 700,000 human games so it learns to imitate people, then let it play itself millions of times to get better. It works. **In a mirror match it beat a World Championships competitor.**

And then they measured how easy it is to beat, and the answer was: completely. They trained a second bot whose only job was to beat the first one, and it did — almost every time. Their human tester said it plainly: *the agent is strong at first, but after enough games in a row, strong players adapt and beat it*. Against a merely *advanced* player it won two games out of five.

We think that is the whole problem, not a bug in their work. Here is the thing to picture. Their bot is a **memorised answer sheet**: for every situation, one answer, worked out in advance and then frozen. That is a great way to play chess, where both players can see everything. It is a terrible way to play poker, where they can't — because once your opponent notices you always fold in a spot, you lose that spot forever.

Pokémon is poker, not chess. You cannot see which four of their six they brought, what they're holding, or their last move. Poker figured this out between 2007 and 2021, and the answer was: **don't memorise, re-think every hand.**

So our headline number changes. We used to ask "how often do we win?" We now ask "**how easily can a prepared opponent read us?**" That is a harder, more honest question, and it is the one their number answers too — so the two projects can be compared even though the bots can never actually play each other. Their score is about 100% readable. **Ours is currently unknown, and saying so is the point** — we've made our headline a number we can't produce yet rather than a number that flatters us.

Our bet — and it is a bet, not a claim. A bot that *recomputes* from scratch every turn should be harder to read than one that *recalls*. There is nothing to memorise about it. Whether that survives in a game where both players move at once, dice are rolled, and the whole thing is over in six turns, **nobody knows. That's the experiment.**

What we do if we're wrong. We use their recipe. It's published, it's open source, and it works. Being wrong here would be a real finding about Pokémon, not a failure.

Slide 9f — We had a number that justified two years of work, and it was wrong by five times (3.62.2)

The claim. We wrote our own Pokémon simulator instead of using the official one, and the reason written down in every document was: **the official one is 117 times slower.** That number decided the architecture.

We measured it again. It's about 25 times slower, not 117.

Why we're telling you rather than quietly fixing it. Three things.

1. The old number stays in the documents, with a dated correction beside it, because this project does not rewrite a past conclusion as though it were never made.
2. **The decision it justified is still right** — 25 times slower is still far too slow to think during a live game. So nothing is being rebuilt.
3. **But the reason has to be a real reason.** "117 times" was doing work it hadn't earned. The honest version is now a claim you could prove us wrong about: *we wrote a fast simulator so the bot can re-think every turn — so the simulator is worth building **if and only if** re-thinking actually helps.* We are about to run the test that decides it.

The part that should worry a reader most. We have three different speed measurements of our own simulator, taken two weeks apart, that disagree by a factor of ten — and **not one test noticed.** We measure win rates to three decimal places and check them against a noise floor. We had never once checked the speed of the thing everything else runs on.

Slide 10 — Read more

Full technical detail, math, and sources: [ABRA white paper](#). Also: [project summary](#) · [technical docs](#) · [model ledger](#).